IMS TRAINING

(INFORMATION MANAGEMENT SYSTEM)

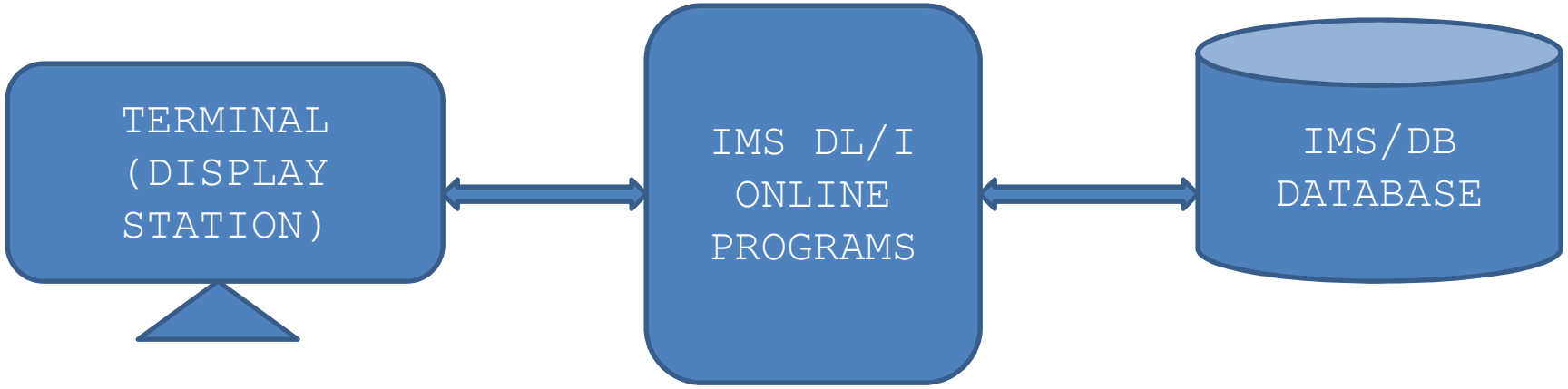
by

M.SIRANJEEVI

© TemplatesWise.com

IMS DC- Data Communication

IMS DC or IMS Data communication is a system that facilitates online transactions through online interactive programs.



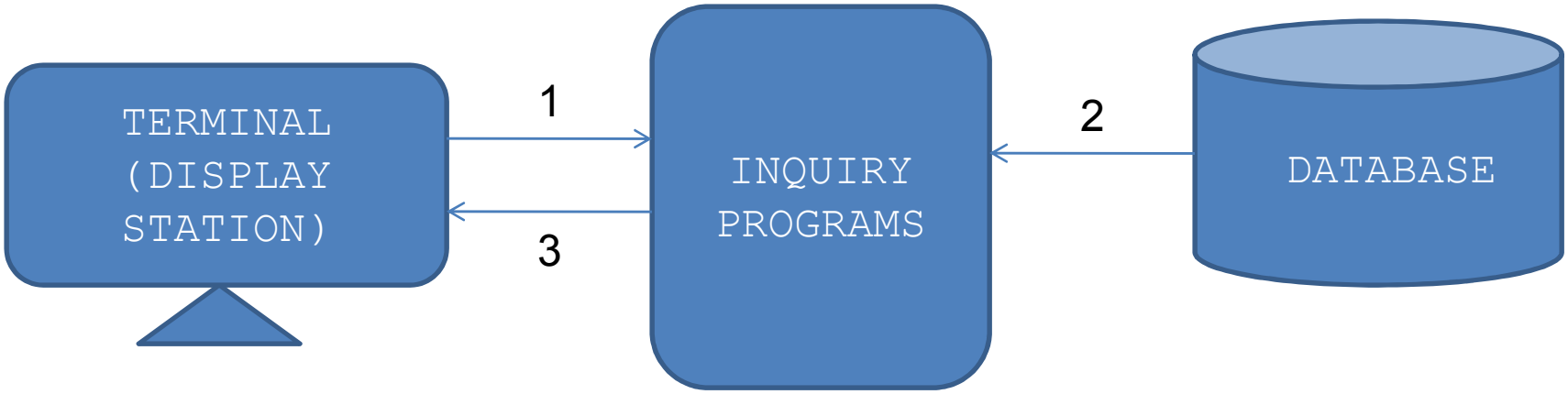
Online programs-Types

There are 4 types of Online programs,

1. Inquiry programs
2. Data entry programs
3. Maintenance programs
4. Menu-driven programs

Inquiry programs

The Inquiry program is designed to respond user's question.

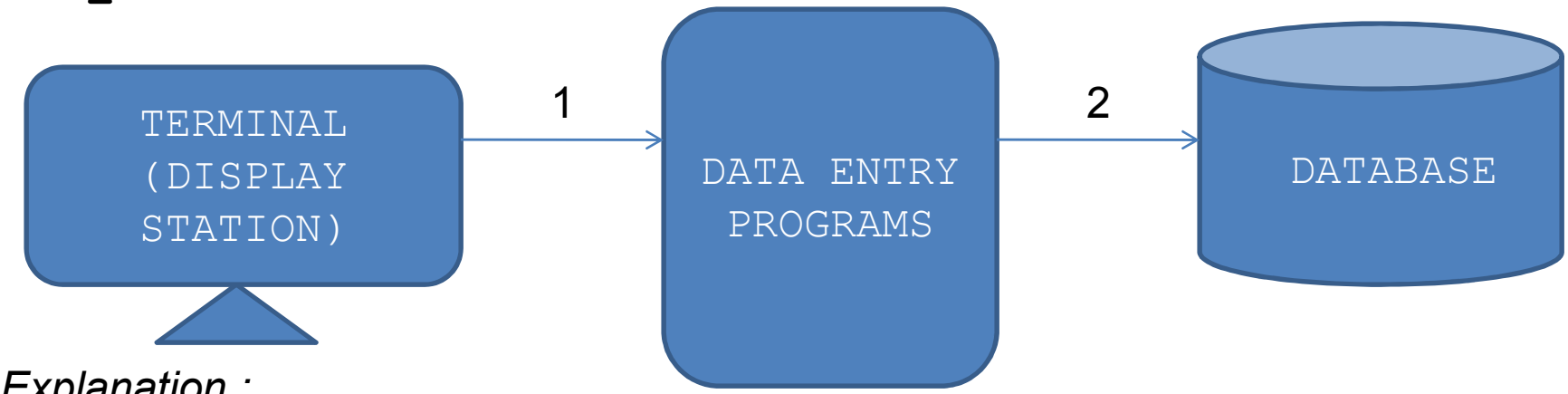


<u>Explanation</u> :

1. *The operator requests the data to be displayed.*
2. *The programs retrieves the appropriate data from the database.*
3. *The program displays the data at the terminal.*

Data entry programs

Although inquiry programs are widely used, they don't let users key in data that's added to the system's database.

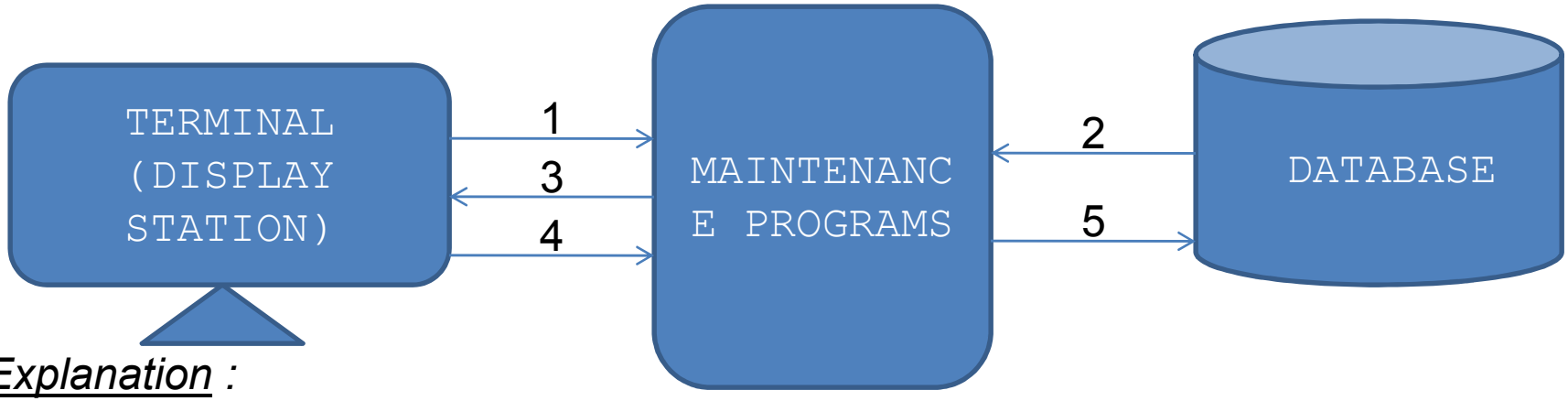


Explanation :

1. *The operator enters the data for one transaction at the terminal.*
2. *The program updates any related databases.*

Maintenance programs

A maintenance program can update a database by adding, replacing and deleting segments.



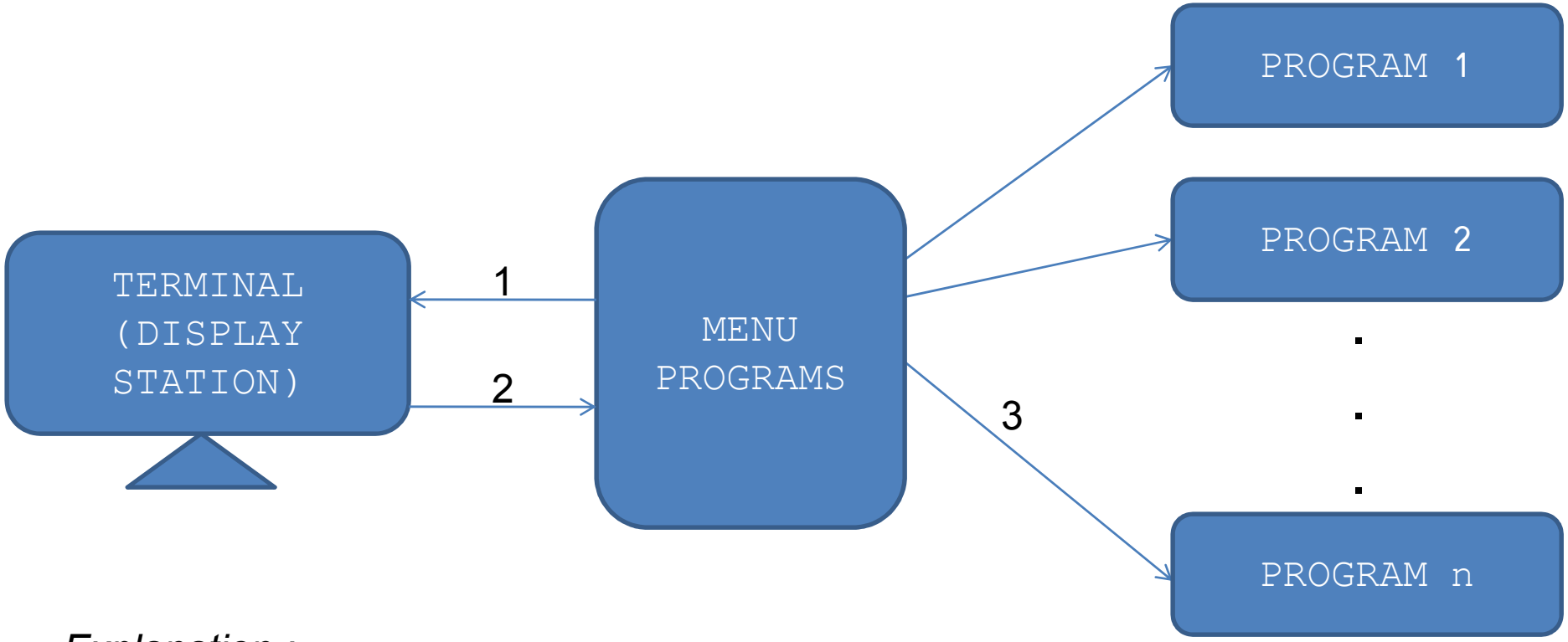
Explanation :

1. *The operator requests the data to be displayed.*
2. *The programs retrieves the appropriate data from the database.*
3. *The program displays the data at the terminal.*
4. *The operator enters the required changes to the data.*
5. *The program rewrites the changes to the database.*

Menu-driven programs

A menu program lets an operator select the functions he wants to perform. An application that's built around a set of menu programs is called *menu-driven systems*. Some interactive programs don't use menus instead operator invokes programs by using explicit commands. This type of method is called *command-driven systems*.

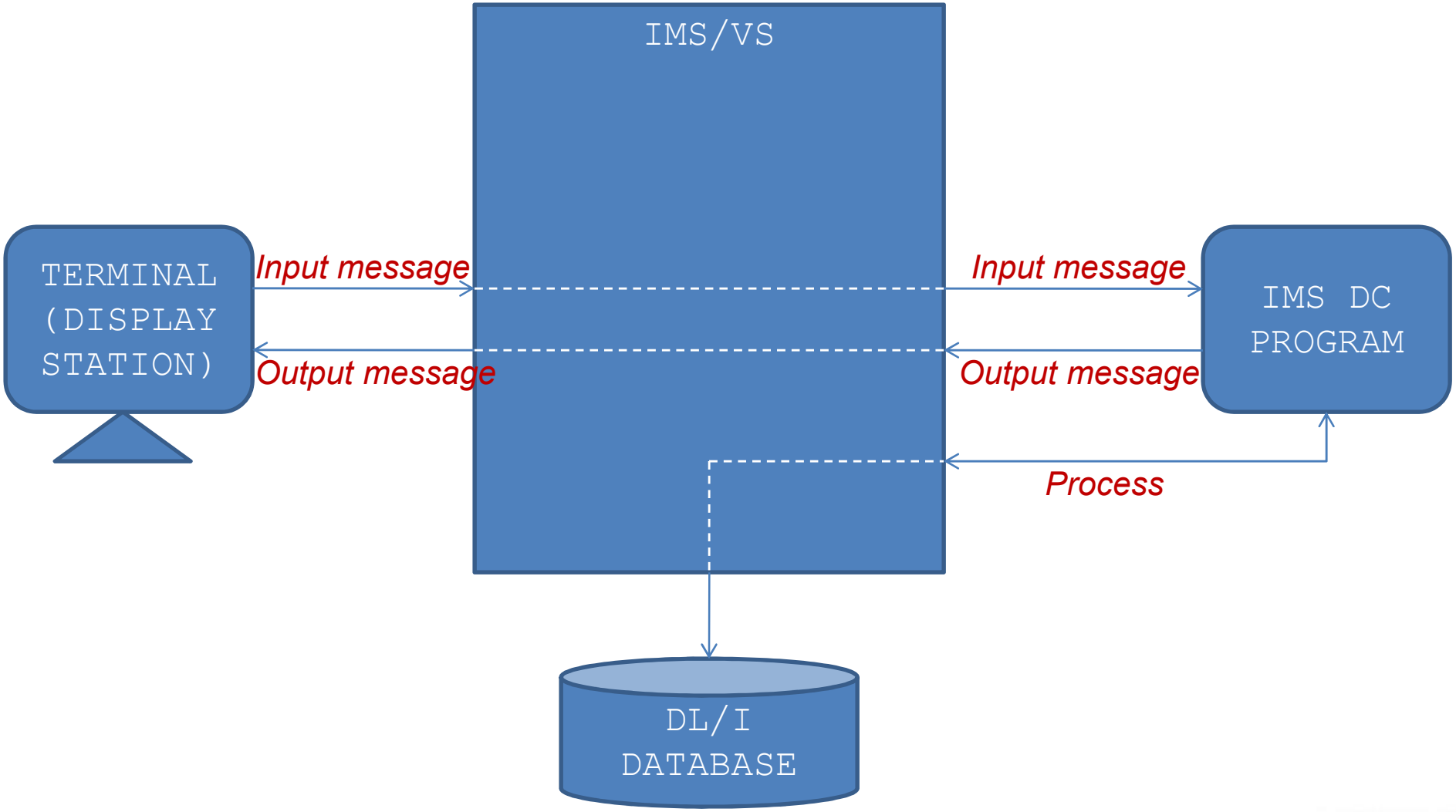
Menu-driven programs



<u>Explanation</u> :

1. *The menu program sends a list of selections to the terminal.*
2. *The operator chooses one of the selections.*
3. *Control is transferred to the appropriate program.*

IMS DC Env. Overview



IMS Address Spaces

- An IMS uses several user regions, each in its own address space. All are managed from one user region called *IMS Control Region*(CTL Region), where the ims control programs resides.
- All database processing and logging are done through the control region.

IMS Address Spaces

- The user regions in the other address spaces ims uses are called *dependent regions*, the region which the IMS DC application programs are executed.
- IMS can manage many dependent regions, there can be upto **255** dependent regions,

IMS Address Spaces

There are two kinds of dependent regions,

1. Message Processing Program Regions (MPP Regions).
2. Batch Message Processing Program Regions (BMP Regions).

MPP Regions

- A *message processing program region* is where IMS schedules and executes message processing programs(pure online programs).
- Most of the dependent regions in an IMS DC system are MPP Regions.
- A system has a fixed number of MPP Regions all are managed by IMS control program running in the control region.

MPP Regions

- Typically a program occupies MPP Region runs until it has processed all the input messages received for it.
- Then it ends and the MPP Region in which it was running is freed and made available for another application program.

BMP Regions

- A batch message processing region is used to execute a kind of IMS DC program called batch message processing programs.
- A BMP Program is much like a MPP, except terminal response is not required for each input message and database processing need not to be done immediately.

Control Region

- All the terminals,databases and logging operations associated with IMS DC programs are performed through the control region. IMS DC has a centralized control over its databases, terminals and logs.
- In addition, control region contains variety of control blocks that are required for data processing.

Terminals and Security under IMS

Logical Terminals :

- For application functions IMS uses logical terminal name(LTERM) rather than physical terminal names.
- Within IMS, each LTERM is associated with one and only physical terminal. IMS knows an LTERM name and associated physical terminal.

Terminals and Security under IMS

- Logical terminal names are limited to **8** characters long and it is assigned by administrator.
- As a DC programmer we dont need to concern LTERM names because IMS keeps track of sources of input messages and automatically causes output messages to be routed back to the originating terminals.

Terminals and Security under IMS

Master Terminal :

- One logical terminal in the IMS network is designated as the master terminal, its a operational center of the system.
- The MTO(Master Terminal Operator) can manage other terminals in the network and program executions in the dependent regions.

Terminals and Security under IMS

- The MTO can also dynamically adjust resources when the system was started.

  For example, if one physical terminal is disabled, the MTO can reassign the LTERM name normally associated with it to another nearby terminal so work can continue.

Terminals and Security under IMS

<u>Security Features</u> :

IMS provides substantial security features that effect both database and terminal use.

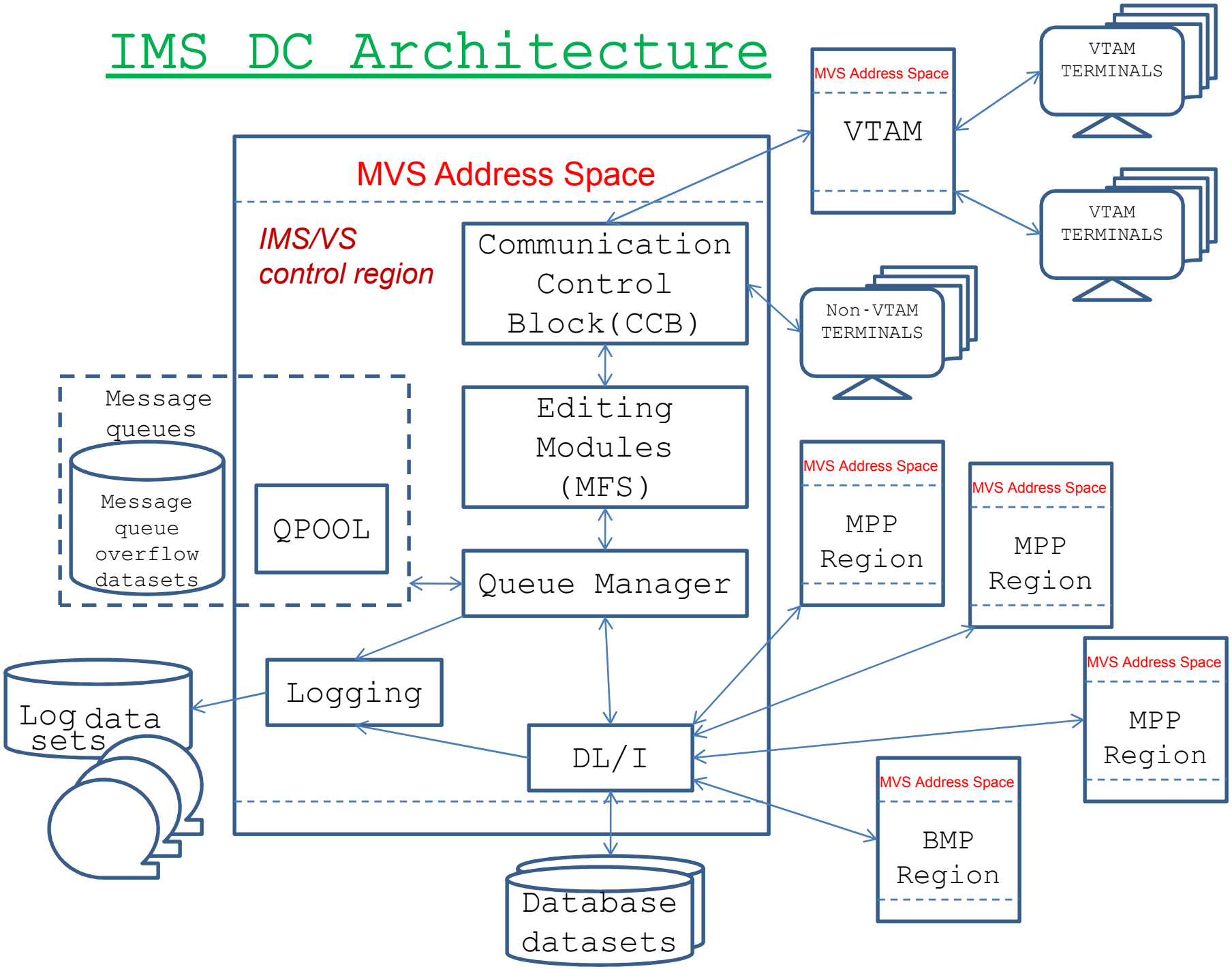
- IMS/VS itself.
- SMU(Security Maintenance Utility)

  - sign-on verification security with transaction authorization.

  - LTERM Security.

  - password security.

IMS DC Architecture



Messages in IMS

Messages are the unit of information that are transferred between terminal and the application programs. There are two types of messages in IMS,

1. Input message.
2. Output message.

Input Message

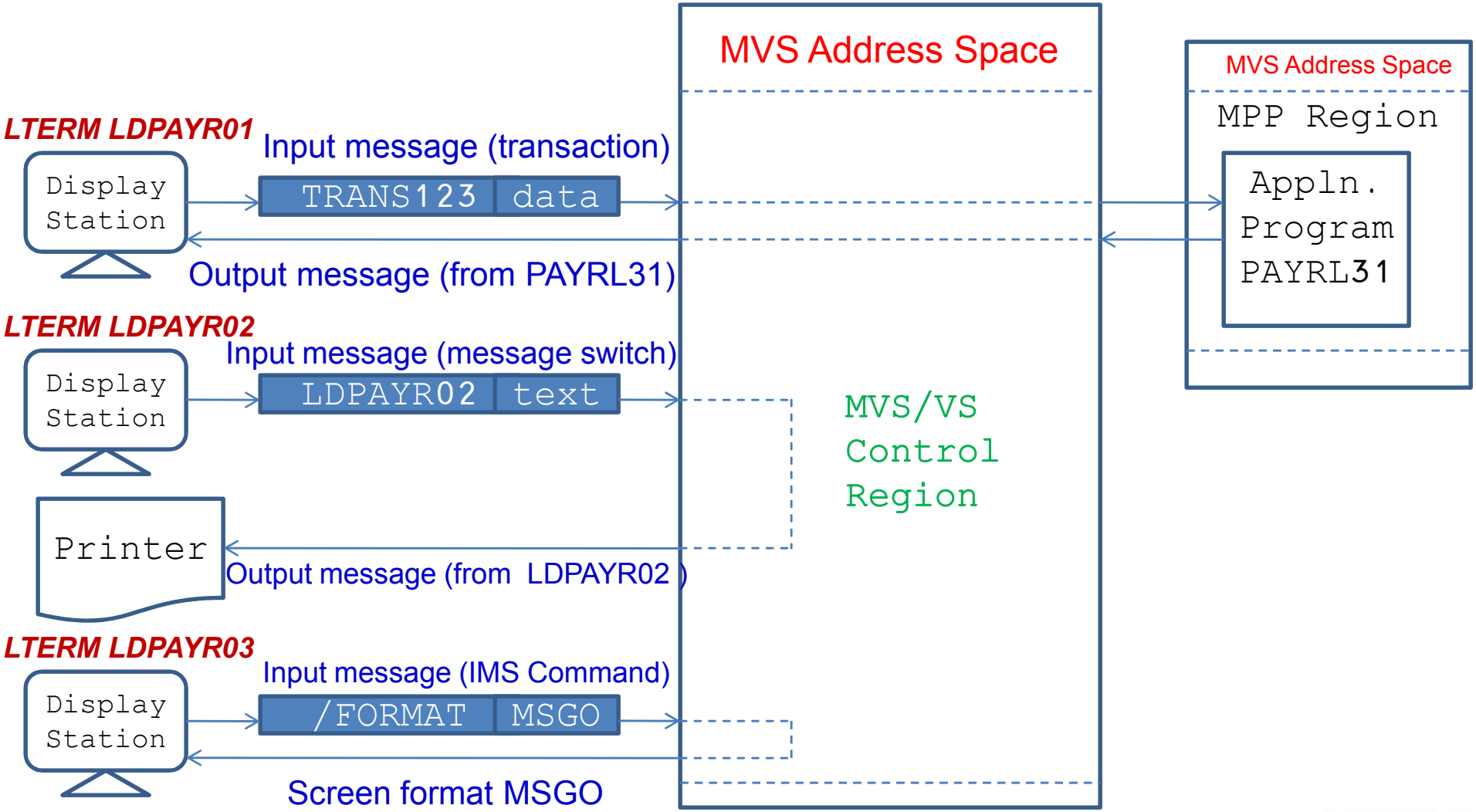
Input message is a unit of information that is transferred from terminal to the application program. There are 3 kinds of input messages,

1. Transactions.
2. Terminal-to-terminal message switches.
3. IMS Commands.

Input Message identification

IMS uses the value in the first eight bytes of an input message to identify the message's type. If the first eight characters name an application program, IMS knows that the message is a transaction code. Similarly if the first eight characters contain an LTERM name, IMS knows that the message is a message switch.

Input Message flow



Input Message editing

When IMS receives a message from a terminal, it is not passed immediately to the appropriate destination. Instead it undergoes editing. Two types of message editing,

1. IMS Basic edit
2. Message Format Service(MFS) edit.

Input Message editing

- Basic edit does little more than remove communication control characters from input messages and add them to output messages.
- MFS provides more extensive editing functions. MFS modules resides in the control region and provide powerful features that exploit the formatted display capabilities to **3270** devices.

Input Message queueing

- Whenever the MFS editing in effect, the transactions aren’t passed directly to their destination application programs after they are edited.
- The entry of terminal user that invokes a transaction and the execution of the application program with which it’s aren’t synchronous.

Input Message queuing

- IMS has to store the transaction temporarily. This is called message queueing.
- IMS modules that perform message queueing are collectively called the queue manager, also called QPOOL.
- The QPOOL is an area of virtual storage within the control region that can be extended as ODS.

Message Format Service (MFS)

- Message Format Service is a very important tool of an IMS DC Programmer. This helps a lot in the extensive editing of the Input Data and also it helps in formatting the Sending and the Receiving Screen for the message, which is being processed. It Dramatically enhance the operator efficiency as well as accuracy of the data.

Message Format Service (MFS)

- MFS Modules reside in IMS Control Region and they are put in action between IMS's communication control module and its queue manager. Because of this, editing is always performed for both the Input and also the Output messages between queuing and also at the communication time.

Message Format Service (MFS)

- IMS DC Programs have some separate control blocks, which control the flow of the Input and Output Message and also assist significantly in designing the screens to send and receive processed messages or transaction.
- In addition to *PSB, PCB, DBD, ACB,* MFS brings into existence, four more control blocks. They are,

MFS Control Blocks

Four Control Blocks,

- DIF - Device Input Format.
- MID - Message Input Descriptor.
- DOF - Device Output Format.
- MOD - Message Output Descriptor.

MFS Control Blocks

For an Input Message, MFS uses:

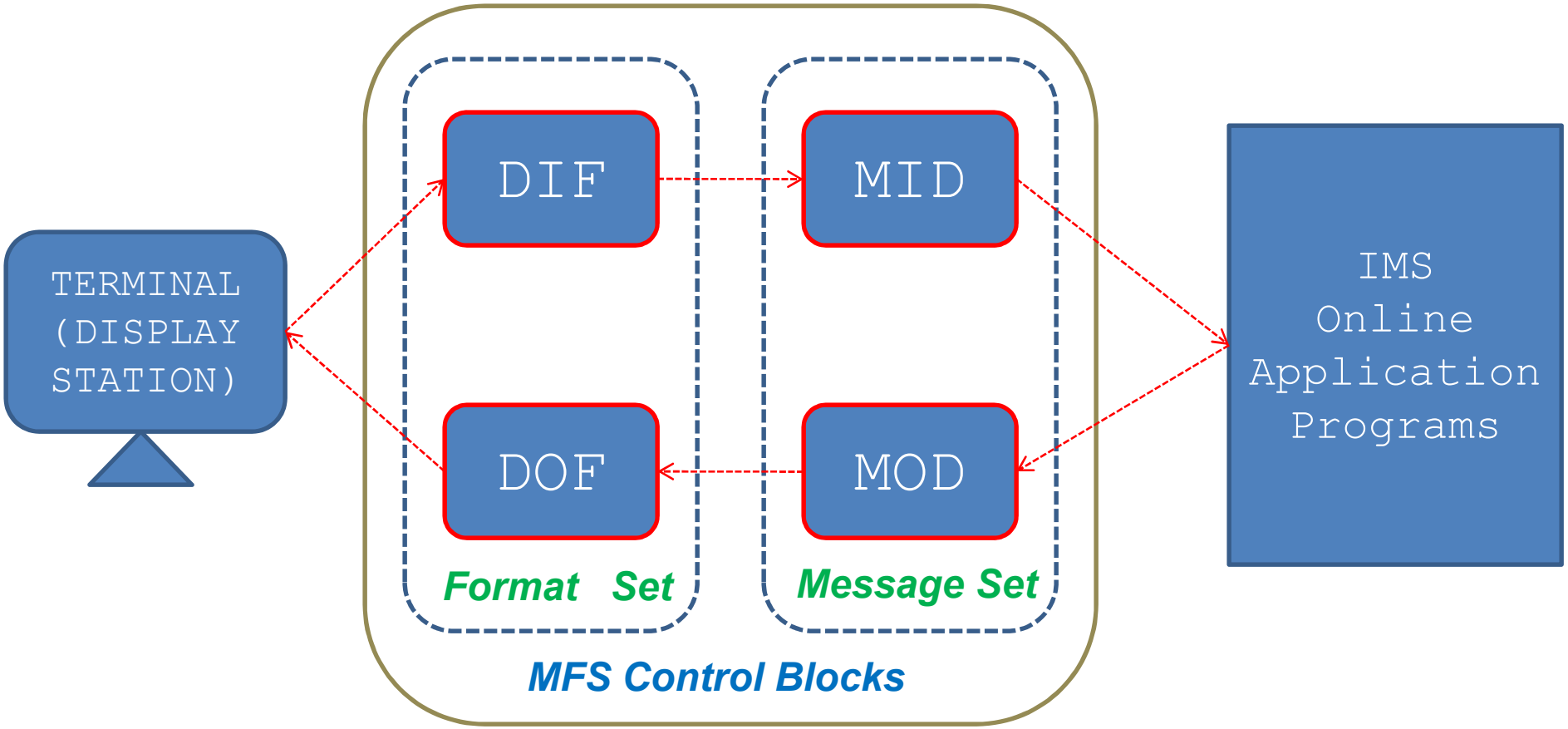
- *Device Input Format(DIF)* to interpret data, as it is received from a user's terminal.
- *Message Input Descriptor(MID)* to determine, how that input data should be formatted and presented to the Application program.

MFS Control Blocks

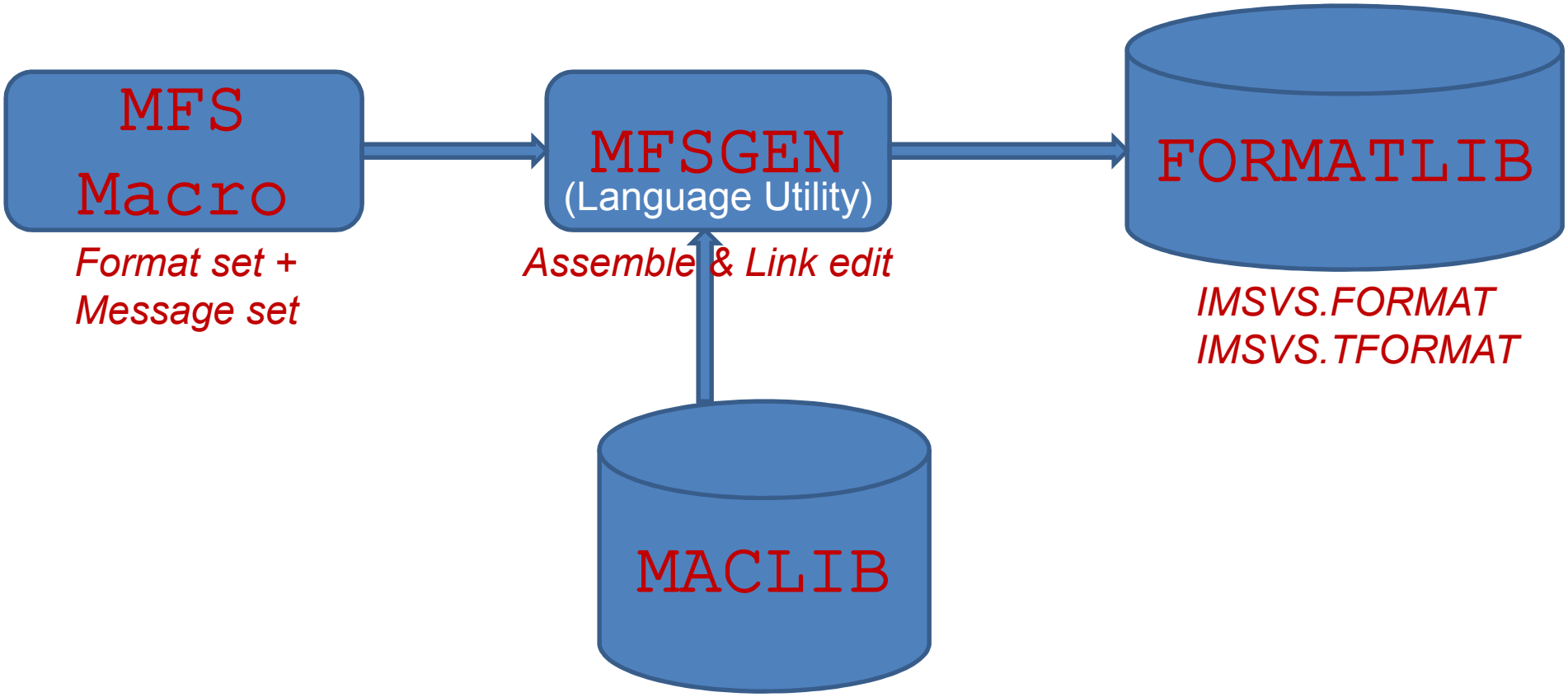
For an Output Message, MFS uses:

- *Message Output Descriptor(MOD)* is used to interpret data received from the proper Application program.
- *Device Output Format(DOF)* is used to determine, how that data should be formatted for transmission to the user's terminal.

MFS Control Blocks



MFS Control Blocks Execution



MFS Coding Rule

- COLUMN 1-8 : LABEL
- COLUMN 10-14 : OPERATION
- COLUMN 16-71 : operand/comment
- COLUMN 72 : CONTINUATION CHAR

LABEL: First character must be alphabetic.

OPERATION: Like FMT,DFLD,MSG.

OPERAND: One or more parameters.

CONTINUATION: Any char. in col 72.

Format Set Hierarchy

```
Label   FMT
            DEV
                DIV
                    DPAGE
                        DFLD
                        ...
        FMTEND
```

Message Set Hierarchy

```
Label   MSG
            SEG
                MFLD
                    . . .
                    . . .
        MSGEND
```

MFS Hierarchy

```
Label   FMT
            DEV
                DIV
                    DPAGE
                          DFLD
                          ...
                          ...
        FMTEND
Label   MSG
            SEG
                MFLD
                    ...
                    ...
        MSGEND
        END
```

Format Set Macros

- FMT
- DEV
- DIV
- DPAGE
- DFLD
- FMTEND

FMT Macro

- FMT stands for Format

  Identifies the beginning of the device format.

```
Label     FMT
          ...
          ...
          FMTEND {End of DF}
```

Label : **1-6** Characters long.

DEV Macro

- DEV stands for DEVICE

  Identifies the beginning of a series of Control Statements for a particular device type. Multiple DEV statements can be coded. The following Parameters are used along with DEV. DEV statement not only identifies a particular kind of device, but it specifies its characteristics as well.

DEV Macro

Parameters: *TYPE,FEAT,PFK,DSCA,SYSMSG.*

TYPE: The TYPE parameter is used to identify the terminal type to which the control Statement that follow apply. DEV TYPE=(Device type,model no.)

3270,A1	12-line,80-col
3270,A2	24-line,80-col
3270,A3	32-line,80-col
3270,A4	43-line,80-col
3270,A5	12-line,40-col
3270,A7	27-line,132-col

DEV Macro

FEAT:

The FEAT Parameter is used to specify the exact configuration of optional features on the above mentioned device for which the DEV statement is coded. Eg: User of Selector Light pens, Magnetic card reader and special data entry keyboards. FEAT=IGNORE

DEV Macro

<u>DSCA:</u> Default System Control Area

The DSCA Parameter lets you specify the default terminal action that can be performed when an output message is sent to the **3270** device.

X'nnnn' IS HEXA DECIMAL VALUE FOR DSCA.

X'**00A0**' Unprotected fields to be erased.

X'**00C0**' Entire device buffer to be cleared.

DSCA=X'**00A0**'/X'**00C0**'

DEV Macro

SYSMSG: System Message

The SYSMSG parameter names a device field in which messages that come to the terminal from IMS will be displayed. The field should be 79 bytes long. SYSMSG=ERRMSG(Max. 8 bytes long of user defined name), the ERRMSG will be defined in the DFLD statements.

DIV Macro

- DIV stands for DIVISION

  This is used to Indicate that the same Device format Specifications should be used to generate both DIF and DOF. Parameter: *TYPE*

  DIV TYPE=INOUT

  It is a default value for all **3270** display stations.

DPAGE Macro

- DPAGE stands for Device Page

  The primary function of this control Statement is to let you specify a device format that will handle multiple logical pages. The other two important functions are initial cursor positioning and field fill.

<u>Parameters:</u> *CURSOR and FILL*

DPAGE Macro

CURSOR:

The Cursor parameter of the DPAGE statement is used to specify a default location where the cursor should be positioned each time a screen is displayed.

```
CURSOR =((5,10))
```

- tells **5**th line,**10**th column

DPAGE Macro

FILL:

The FILL parameter FILL=*Fill_char* this parameter will specify what character should be used to pad device fields when output data sent to them doesn't fill them.

FILL=PT/X'hh'/C'c'

-Default value is PT.

DPAGE Macro

FILL:

Fill_char:

- PT: Program tab(the default) output field do not fill the device field are followed by a program tab character to erase of the device field.
- C'c': specifies character to be filled
- X'hh': specifies a hexadecimal value to be used in fill char.

DFLD Macro

- DFLD: stands for Device Field

  This will specify where on the screen a field is located and how large the field is (the attributes)

DFLD with labels : only when the device field has to be related to a message field (for input, output, or both)

DFLD without labels : which are known as CAPTIONS

DFLD Macro

<u>Parameters:</u> *'Literal',POS,LTH,ATTR,EATTR*

```
Label  DFLD  ['literal'],
             POS=(line,column),[LTH=length],
             [ATTR=(ALPHA | NUM ,
                    NOPROT| PROT,
                    NORM | NODISP | HI,
                    NOMOD | MOD)]
             [EATTR=(HD | HUL ,
                     CD | BLUE | RED | PINK |
                     GREEN | TURQ | YELLOW)]
```

Message Set Macros

- MSG
- SEG
- MFLD
- MSGEND
- END

MSG Macro

- MSG stands for Message

  Identifies the beginning of message descriptor.

```
Label     MSG   Parameters
          ...
          ...
          MSGEND {End of MD}
```

Label : 1-8 Characters long.

MSG Macro

<u>Parameters:</u> *TYPE,SOR,NXT*

<u>TYPE:</u> TYPE parameter is used to specify whether a MID or MOD has to be generated. TYPE=(INPUT/OUTPUT)

- Unlike the situation for a DIF and DOF you must code separate groups of statements to define a MID and a MOD. Each segment of code that describes a message, whether it's an input message or an output message, begins with a MSG statement and ends with a MSGEND stmt.

MSG Macro

SOR: stands for Source

SOR parameter is required to define the name of the device format (i.e. label of the DEV statement).

SOR=(FMTname,IGNORE)

NXT: stands for Next

NXT parameter specifies the message descriptor, MFS will use during execution after it uses the one defined by the statements associated with the MSG stmt. NXT=Next_msg_name

SEG Macro

- SEG stands for Segment

  Identify the segments that make up the messages. This macro is the starting point of creating a segment or number of segments in MID or MOD. A Segment (SEG) consists of message or messages. Messages can be created by using MFLD macro. It doesn’t has parameters.

MFLD Macro

- MFLD stands for Message Field

  This will describe the details of the message fields of both input and output.

```
Label  MFLD {source/destination}
            ,LTH=length,JUST=L/R
            ,FILL=Null/X'hh'/C'c'
            ,ATTR=NO/YES
```

MFLD Macro

SOURCE: literal/devfield

DESTINATION: devfield/(devfield,syslit)

System Literals:

Name	Length	Format
DATE1	6	YY.DDD
DATE2	8	MM/DD/YY
DATE3	8	DD/MM/YY
DATE4	8	YY/MM/DD
TIME	8	HH:MM:SS
LTNAME	8	Logical Terminal Name

SEGMENT & MESSAGE

SEGMENT - The unit of access; for the DC system, the smallest amount of data that can be transferred by one IMS operation.

MESSAGE - A unit of data that is transmitted between a program and a terminal.

Sample Screen

```
                    ADDITION SCREEN

                                                    DATE:
                                                    TIME:
                                                    LTNAME:
        ENTER NO1  :

        ENTER NO2  :

        RESULT     :

        MESSAGE    :
```

Sample Code

```
ADDFMT   FMT
         DEV   TYPE=(3270,2),FEAT=IGNORE,DSCA=X'00C0',SYSMSG=ERRMSG
         DIV   TYPE=INOUT
         DPAGE CURSOR=((10,20)),FILL=PT
         DFLD  'ADDITION SCREEN', POS=(4,25),ATTR=(PROT,HI)
         DFLD  ' DATE: ', POS=(6,40),ATTR=PROT
DAT      DFLD  POS=(6,50),LTH=8,ATTR=PROT
         DFLD  ' TIME: ', POS=(7,40),ATTR=PROT
TIM      DFLD  POS=(7,50),LTH=8,ATTR=PROT
         DFLD  ' LTNAME: ', POS=(8,40),ATTR=PROT
LTN      DFLD  POS=(8,50),LTH=8,ATTR=PROT
         DFLD  ' ENTER NO1: ', POS=(10,5),ATTR=PROT
NO1      DFLD  POS=(10,20),LTH=3,ATTR=(NOPROT,MOD)
         DFLD  ' ENTER NO2: ', POS=(12,5),ATTR=PROT
NO2      DFLD  POS=(12,20),LTH=3,ATTR=(NOPROT,MOD)
         DFLD  ' RESULT: ', POS=(15,5),ATTR=PROT
RES      DFLD  POS=(15,20),LTH=5,ATTR=PROT
         DFLD  'MESSAGE: ',POS=(20,5),ATTR=(PROT,HI)
MSG      DFLD  POS=(20,20),LTH=50,ATTR=PROT
ERRMSG   DFLD  POS=(22,1),LTH=79,ATTR=PROT
         FMTEND
```

Sample Code

```
MSGI      MSG    TYPE=INPUT,SOR=ADDFMT,NXT=MSGO
          SEG
          MFLD   'COBOL55',LTH=8
          MFLD   NO1,LTH=3
          MFLD   NO2,LTH=3
          MSGEND

MSGO      MSG    TYPE=OUTPUT,SOR=ADDFMT,NXT=MSGI
          SEG
          MFLD   NO1,LTH=3
          MFLD   NO2,LTH=3
          MFLD   RES,LTH=4
          MFLD   MSG,LTH=50
          MFLD   ERRMSG,LTH=79
          MFLD   (DAT,DATE2)
          MFLD   (TIM,TIME)
          MFLD   (LTN,LTNAME)
          MSGEND

          END
```

Programming elements for DC Programming

- DL/I calls for DC operations.
- the PCB mask for DC operations.
- how to retrieve input message segments.
- how to send output message segments.

Programming elements for DC Programming

- <u>DL/I calls for DC operations:</u>

✓ Just like a database call, a basic unit of I/O for data communication DL/I call is a segment.

✓ Most messages consists of one segment occurrence or multiple segment occurrences, not to be arranged like database calls.

Programming elements for DC Programming

- <u>DL/I function in DC call:</u>

✓ Just like a database call, the first argument you code in DC call is the function code is of **4** bytes long. The function codes applicable to IMS DC Calls,

- ✓ ISRT
- ✓ GU
- ✓ GN

Programming elements for DC Programming

- ✓ <u>ISRT:</u>

  To send each segment from application program to the display station.

- ✓ <u>GU:</u>

  GU call will retrieve the very first segment of an input message.

Programming elements for DC Programming

- ✓ <u>GN:</u>

  If an input message consists of multiple message segments then the subsequent messages will be retrieved by using GN call.

ISRT	Send
GU	Receive(First segment only)
GN	Receive(Subsequent segments)

Programming elements for DC Programming

- <u>The PCB mask for DC operation:</u>

```
01     IO-PCB-MASK.
   05  IO-PCB-LOGICAL-TERMINAL  PIC  X(8).
   05  FILLER                   PIC  XX.
   05  IO-PCB-STATUS-CODE       PIC  XX.
   05  IO-PCB-DATE              PIC  S9(7)     COMP-3.
   05  IO-PCB-TIME              PIC  S9(6)V9   COMP-3.
   05  IO-PCB-MSG-SEQ-NUMBER    PIC  S9(5)     COMP.
   05  IO-PCB-MOD-NAME          PIC  X(8).
   05  IO-PCB-USER-ID           PIC  X(8).
```

Programming elements for DC Programming

- Segment I/O area in a DC call:

✓ Input message I/O area format:

```
01    INPUT-AREA.
   05  LL          PIC  S9(4)  COMP.
   05  ZZ          PIC  XX.
   05  TRANS-ID    PIC  X(8).
   05  USER-DATA   PIC  X(n).
```

Programming elements for DC Programming

- Segment I/O area in a DC call:

✓ Output message I/O area format:

```
01    OUTPUT-AREA.
   05  LL          PIC  S9(4)  COMP.
   05  Z1          PIC  X.
   05  Z2          PIC  X.
   05  USER-DATA   PIC  X(n).
```

Programming elements for DC Programming

- ✓ <u>ISRT:</u>

```
CALL 'CBLTDLI' USING  'ISRT'
                      IO-PCB-MASK
                      OUTPUT-AREA.
```

- ✓ <u>GU/GN:</u>

```
CALL 'CBLTDLI' USING  'GU/GN'
                      IO-PCB-MASK
                      INPUT-AREA.
```

Sample program

```
IDENTIFICATION DIVISION.
PROGRAM-ID. ADDPGM.
DATA DIVISION.
WORKING-STORAGE SECTION.
01    INPUT-AREA.
     05  LL          PIC  S9(4)  COMP.
     05  ZZ          PIC  XX.
     05  TRANS-ID    PIC  X(8).
     05  USER-DATA.
           10  NO1I  PIC  999.
           10  NO2I  PIC  999.
01    OUTPUT-AREA.
     05  LL1         PIC  S9(4)  COMP.
     05  Z1          PIC  X.
     05  Z2          PIC  X.
     05  USER-DATA.
           10  NO1O  PIC  999.
           10  NO2O  PIC  999.
           10  RES   PIC  9999.
           10  MSG   PIC  X(50).
```

Sample program

```
01  FUN1  PIC  X(4)  VALUE  'ISRT'.
01  FUN2  PIC  X(4)  VALUE  'GU'.
LINKAGE SECTION.
01    IO-PCB-MASK.
     05  IO-PCB-LOGICAL-TERMINAL  PIC  X(8).
     05  FILLER                   PIC  XX.
     05  IO-PCB-STATUS-CODE       PIC  XX.
     05  IO-PCB-DATE              PIC  S9(7)    COMP-3.
     05  IO-PCB-TIME              PIC  S9(6)V9  COMP-3.
     05  IO-PCB-MSG-SEQ-NUMBER    PIC  S9(5)    COMP.
     05  IO-PCB-MOD-NAME          PIC  X(8).
     05  IO-PCB-USER-ID           PIC  X(8).
 PROCEDURE DIVISION.
     ENTRY  'DLITCBL'  USING   IO-PCB-MASK.
     CALL   'CBLTDLI'  USING   FUN2
                               IO-PCB-MASK
                               INPUT-AREA.
     MOVE  NO1I  TO  NO1O.
     MOVE  NO2I  TO  NO2O.
```

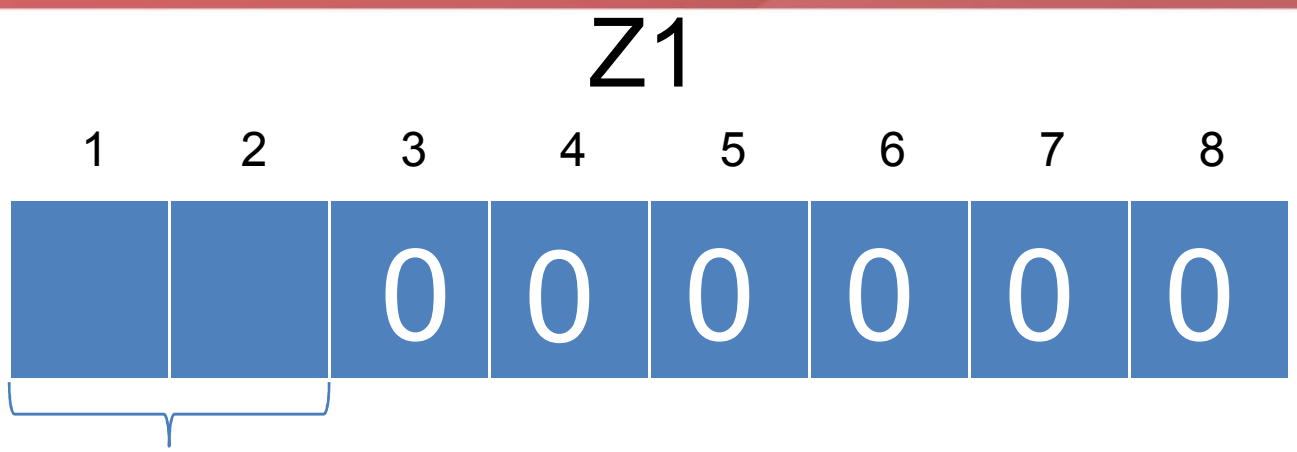
Sample program

```
COMPUTE  RES  =  NO1I  +  NO2I.
MOVE  'ADDITION IS SUCCESSFUL'  TO  MSG.
MOVE  64  TO  LL1.
CALL   'CBLTDLI'  USING   FUN1
                          IO-PCB-MASK
                          OUTPUT-AREA.
GOBACK.
```

External PCB Mask:

```
PCB   TYPE=TP,LTERM=LTERM123
PSBGEN  LANG=COBOL,PSBNAME=EIOPCB
END
```

Dynamic cursor positioning



0	0	Cursor positioning not requested.
1	1	Position the cursor at this field.

Note: Bits 0 and 1 must have the same value; value 01 and 10 are invalid.

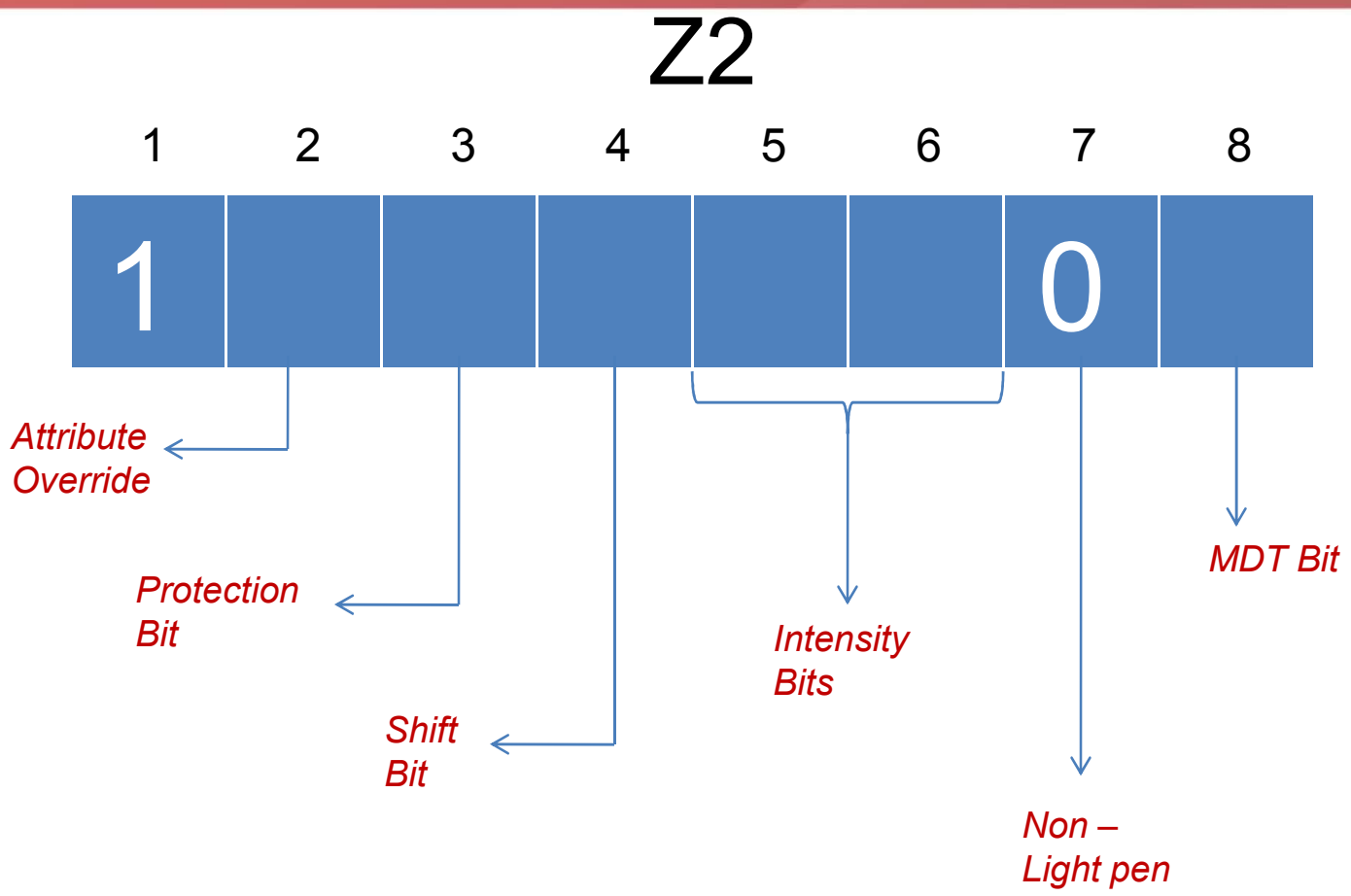
Dynamic cursor positioning

<u>Default:</u>

X'00' - Dynamic cursor positioning not requested.

X'C0' - Dynamic cursor positioning requested.

Dynamic attribute change



Dynamic attribute change

Attribute override:

0 - Activate only specified attributes.

1 - Activate all attributes.

Protection bit:

0 - Unprotected.

1 - Protected.

Dynamic attribute change

Shift bit:

0 - Alphanumeric shift.

1 - Numeric shift.

Intensity bits:

00 Normal intensity.

01 No display.

10

11

Bright intensity.

Dynamic attribute change

Light pen detectability bit:

0 - Non light pen applications.

Modified data tag:

0 - MDT off.

1 - MDT on.

Dynamic attribute change

X’80’ - Default.

X’A0’ - Unprotected to Protected.

X’90’ - Alpha shift to Numeric shift.

X’84’ - Intensity - No display.

X’88’ - Intensity – Bright.

X’81’ - MDT ON.

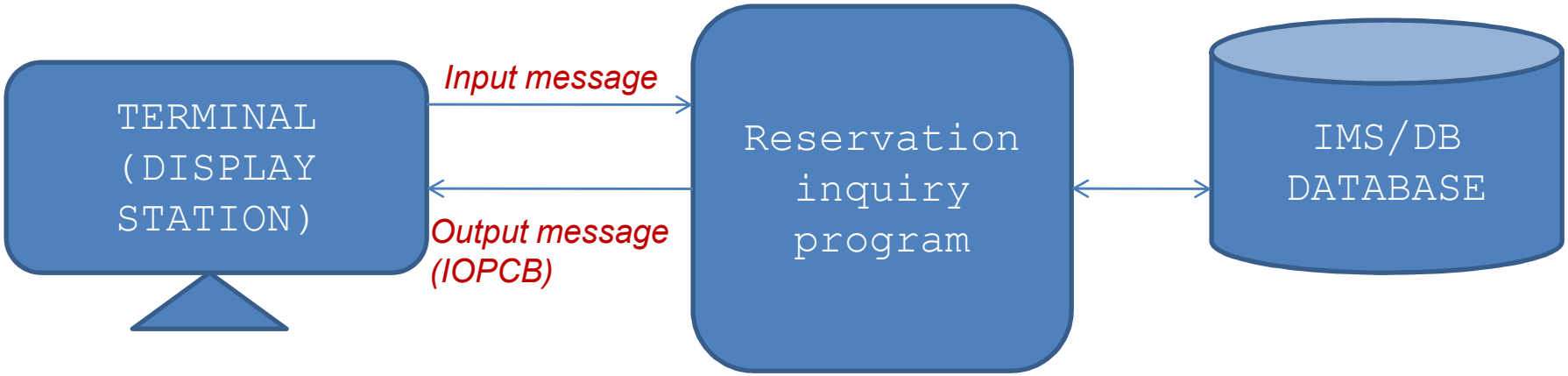
Alternate Destinations

The alternate destination facility of IMS is flexible. An application program that's processing a single input message can send multiple output messages to multiple destinations.

The application program issues an ISRT call that's bound to originating terminal. The difference is ISRT call doesn't specify I/O PCB, but rather of an *Alternate PCB.*

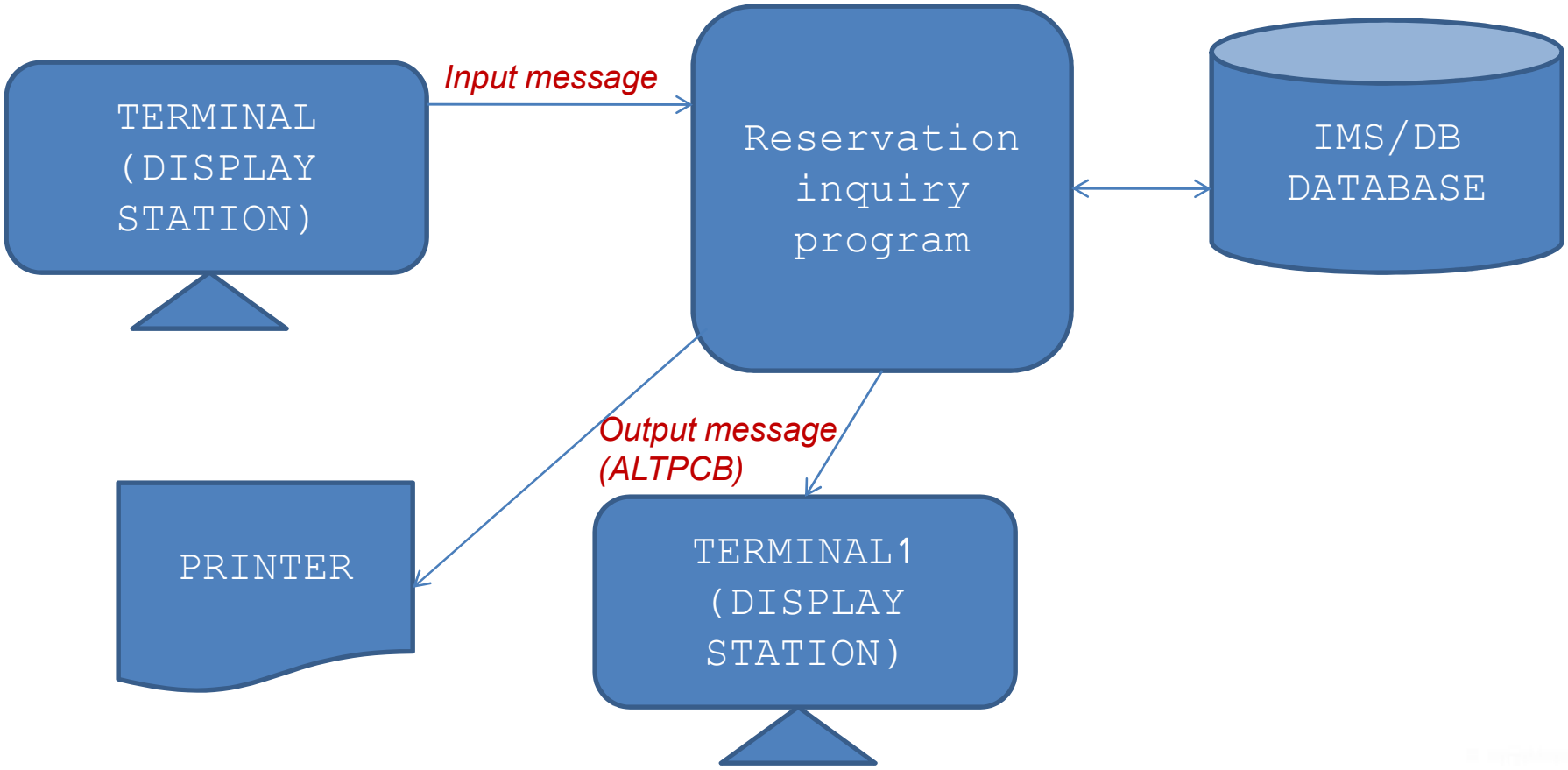
Alternate Destinations

A simple online reservation program flow with I/O PCB :



Alternate Destinations

A simple online reservation program flow with Alternate PCB :



Alternate Destinations

Alternate PCB Mask structure:

```
01    ALT-PCB-MASK.
      05  ALT-PCB-DESTINATION   PIC  X(8).
      05  FILLER                PIC  XX.
      05  ALT-PCB-STATUS-CODE   PIC  XX.
```

ENTRY statement lists all the PCB's that program uses, The sequence will be : ENTRY 'DLITCBL' USING

I/O PCB, ALTERNATE PCB, INTERNAL PCB.

Alternate Destinations

Application program uses Alternate PCB:

```
CALL 'CBLTDLI' USING DLI-ISRT
                     ALT-PCB-MASK
                     OUTPUT-AREA.
```

Alternate Destinations

Alternate PCB Types,

- ✓ FIXED ALTERNATE PCB
- ✓ MODIFIABLE ALTERNATE PCB

When the administrator defines the Alternate PCB, he specifies whether it’s Fixed alternate PCB or Modifiable PCB.

Alternate Destinations

If it's *Fixed Alternate PCB*,

An application program cannot change the terminal or program associated with fixed alternate PCB.

```
PCB TYPE=TP,LTERM=LTERM123,MODIFY=NO
```

```
CALL 'CBLTDLI' USING DLI-ISRT
                     ALT-PCB-MASK
                     OUTPUT-AREA.
```

Alternate Destinations

If it's *Modifiable Alternate PCB*,

An application program can change the terminal or program to which associated output messages are sent.

PCB TYPE=TP,LTERM=LTERM123,MODIFY=YES

We can achieve the modifiable alternate PCB through two advanced function codes,

- CHNG
- PURG

Alternate Destinations

- The CHNG call tells IMS to store the name of a destination you supply in the modifiable alternate PCB so we can issue ISRT call against it and have the messages routed properly.

```
 01 WS-DESTINATION-NAME PIC X(8).
CALL 'CBLTDLI' USING DLI-CHNG
                     ALT-PCB-MASK
                     WS-DESTINATION-NAME.
```

Alternate Destinations

- The PURG call tells IMS that the message we have been building through alternate PCB is complete and we want to build another message for another destination.

```
CALL 'CBLTDLI' USING DLI-PURG
                     ALT-PCB-MASK.
```

Alternate Destinations

```
MOVE 'LTERM2' TO WS-DESTINATION-NAME.
 CALL 'CBLTDLI' USING DLI-CHNG ALT-PCB-MASK
                                WS-DESTINATION-NAME.
 CALL 'CBLTDLI' USING DLI-ISRT ALT-PCB-MASK
                                          OUTPUT-AREA.
 CALL 'CBLTDLI' USING DLI-PURG ALT-PCB-MASK.

MOVE 'LTERM3' TO WS-DESTINATION-NAME.
 CALL 'CBLTDLI' USING DLI-CHNG ALT-PCB-MASK
                                WS-DESTINATION-NAME.
 CALL 'CBLTDLI' USING DLI-ISRT ALT-PCB-MASK
                                          OUTPUT-AREA.
 CALL 'CBLTDLI' USING DLI-PURG ALT-PCB-MASK.
```

Alternate Destinations

```
MOVE 'LTERM2' TO WS-DESTINATION-NAME.
 CALL 'CBLTDLI' USING DLI-CHNG ALT-PCB-MASK
                                WS-DESTINATION-NAME.
 CALL 'CBLTDLI' USING DLI-GU ALT-PCB-MASK
                                          INPUT-AREA.
 CALL 'CBLTDLI' USING DLI-PURG ALT-PCB-MASK.

MOVE 'LTERM3' TO WS-DESTINATION-NAME.
 CALL 'CBLTDLI' USING DLI-CHNG ALT-PCB-MASK
                                WS-DESTINATION-NAME.
 CALL 'CBLTDLI' USING DLI-ISRT ALT-PCB-MASK
                                          OUTPUT-AREA.
 CALL 'CBLTDLI' USING DLI-PURG ALT-PCB-MASK.
```

Conversational Programs

A Conversational program uses special IMS facilities intended specifically for maintaining data between terminal interactions. We can use **3** techniques to maintain the data between terminal interactions,

- Store data on the terminal screen.
- Store data in a database.
- IMS Scratch Pad Area(SPA).

Conversational Programs

Scratch Pad Area(SPA):

This is the special area provided by IMS to store the data between the terminal interactions.

If a program uses a SPA, It's a 'Conversational' program. It can carry continuous conversations with the terminal user even if it processes the transactions from other users between the terminals.

Conversational Programs

A unique SPA is associated with each terminal that's running a conversational transaction.

SPA STRUCTURE:

```
01  SPA-IO-AREA.
    05  SPA-LL  PIC  S9(4) COMP.
    05  SPA-ZZ  PIC  S9(4) COMP.
    05  SPA-CI  PIC  S9(4) COMP.
    05  SPA-TRANSACTION-CODE  PIC X(8).
    05  SPA-UER-DATA  PIC X(n).
```

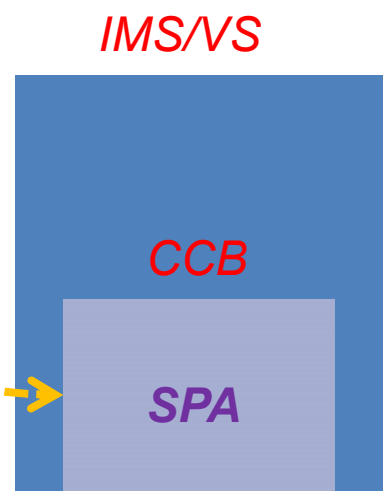
Conversational Programs

In a conversational program we should retrieve the SPA Area before retrieve the message segments. To do that we have to issue GU call against the I/O PCB.

```
CALL 'CBLTDLI' USING  DLI-GU
                      IO-PCB-MASK
                      SPA-IO-AREA.
```

Conversational Programs

```
01  SPA-IO-AREA.
    05  SPA-LL  PIC  S9(4) COMP.
    05  SPA-ZZ  PIC  S9(4) COMP.
    05  SPA-CI  PIC  S9(4) COMP.
    05  SPA-TRANSACTION-CODE  PIC X(8).
    05  SPA-UER-DATA  PIC X(n).
```



THE

END